05 / 26 — Phase 5 Assessment: Design Patterns

Covers: lessons 01 through 25 of Phase 5. **Prerequisites:** all of Phase 5, plus the Phase 3 and Phase 4 material those lessons link to. **Format:** closed-notes first attempt, then open-notes review against the answer key. Allow 4–5 hours. Write answers (and code) before reading Part 7.

---

1. How this assessment works

1.1 Rules

1. Attempt every question without reading the answer key. Use an editor with TypeScript checking and a scratch React project (Vite + TypeScript, React 19.x) for implementation tasks, but do not copy from the lessons.
2. For code tasks, make the code compile under `strict` TypeScript and be able to explain each line.
3. For prediction tasks, **write the prediction before running anything**. Then run it; if the result differs, explain why your model was wrong before moving on.
4. Time-box: Part 2 (explanation) 45 min, Part 3 (prediction) 30 min, Part 4 (debugging) 50 min, Part 5 (implementation) 90 min, Part 6 (architecture) 45 min.
5. Score yourself with the rubric in Part 8, honestly. Partial credit is defined per question.

1.2 Categories and points

Category	Where	Points
Explanation (conceptual)	Part 2: E1–E14 (2 points each)	28
Reasoning (prediction + architecture)	Part 3: P1–P6 (2 each = 12); Part 6: A1–A3 (4 each = 12)	24
Debugging	Part 4: D1–D4 (6 each)	24
Implementation	Part 5: I1–I4 (6 each)	24
Total		**100**

Pass requires **≥ 80 overall and ≥ 70 % in each category** (see Part 8). Version baseline: React 19.x, TypeScript 5.x. Items that depend on React-version details are flagged ⚠; verify against the current docs when in doubt.

---

2. Part 2 — Conceptual questions (Explanation, 28 points)

Answer each in 3–6 sentences. Two points each: 2 = correct and gives the *causal reason*; 1 = mostly correct but names the rule without the reason, or misses an important qualification; 0 = incorrect.

E1. (Lesson 01) Pick two SOLID principles and show, with a React example for each, what a *violation* looks like and what its practical cost is. Then name one place where applying a SOLID principle literally would be over-engineering.

E2. (Lesson 02) Explain the "wrong abstraction" problem. Why is duplication sometimes cheaper than a shared component, and what signals tell you that duplication has become worth removing?

E3. (Lesson 03) What does React's guidance on composition vs inheritance actually claim? Give two composition mechanisms for customizing a component and one situation where each fits best.

E4. (Lessons 04, 18) Container/presentational originated when function components could not hold state. What problem does the *separation* still solve today, and in which situations is the split not worth it? How does a server/client component boundary relate?

E5. (Lesson 05) Why do compound components communicate through context instead of the parent inspecting `children`? Name two ways child inspection breaks.

E6. (Lesson 06) "Custom hooks share state between components." Is that true? Explain what hooks share and what each call site gets. Give a consequence for API design (for example, what to return).

E7. (Lessons 07, 18) What is a headless component/hook, what does it give up compared with a styled component, and when is that trade not worth it? Include the role of accessibility behavior.

E8. (Lesson 08) Explain why an inline context value (`value={{ a, b }}`) can cause avoidable re-renders and how splitting contexts or memoizing changes that. Why should the consumer hook usually throw when the provider is missing?

E9. (Lesson 09) State three concrete drawbacks of HOCs and one case where a HOC is still a reasonable choice.

E10. (Lesson 10) Compare render props and custom hooks for sharing behavior. When is a render prop still the only good option?

E11. (Lesson 11) Define controlled vs uncontrolled components. What happens (and what warning appears) if a component switches between them, and how should a reusable component handle both modes?

E12. (Lesson 12) Why must reducers be pure? Describe one development-time behavior of React that exposes impure reducers.

E13. (Lesson 13) Why does modeling a workflow as a finite state machine (or discriminated union + transition function) prevent bugs that boolean flags allow? What does it *not* prevent?

E14. (Lesson 14) What problem does `useSyncExternalStore` solve that `useEffect` + `useState` subscription code does not (think: concurrent rendering)? What does `getSnapshot` have to guarantee?

---

3. Part 3 — Code prediction (Reasoning, 12 points)

For each, predict the output/behavior and explain the cause in 2–4 sentences. Two points each: 1 for the correct prediction, 1 for the correct causal explanation. Assume React 19, `createRoot`, and **development mode with** `<StrictMode>` unless stated otherwise. State separately what differs in production.

P1.

```
function App() {
  return <Clock><Expensive /></Clock>;
}

function Clock({ children }: { children: React.ReactNode }) {
  const [t, setT] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setT((x) => x + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <div>{t}{children}</div>;
}

function Expensive() {
  console.log("Expensive render");
  return <p>static</p>;
}
```

How many times is `"Expensive render"` logged during the first 5 seconds (dev StrictMode)? In production?

P2.

```
type S = { items: string[] };
function reducer(state: S, action: { type: "add"; item: string }): S {
  state.items.push(action.item);
  return { items: state.items };
}
function List() {
  const [s, dispatch] = useReducer(reducer, { items: [] });
  return (
    <>
      <button onClick={() => dispatch({ type: "add", item: "a" })}>add</button>
      <ul>{s.items.map((x, i) => <li key={i}>{x}</li>)}</ul>
    </>
  );
}
```

After one click, how many `<li>` in development StrictMode? In production? Why?

P3.

```
function Name() {
  const [name, setName] = useState<string>();
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

What happens when the user types the first character? What does the console show?

P4.

```
const store = { width: window.innerWidth };
function useSize() {
  return useSyncExternalStore(
    (cb) => { window.addEventListener("resize", cb); return () =>
window.removeEventListener("resize", cb); },
    () => ({ width: window.innerWidth }),
  );
}
```

What goes wrong, and what does React do about it in development?

P5.

```
function Layout({ children }: { children: React.ReactNode }) {
  const kids = Children.toArray(children) as React.ReactElement[];
  const header = kids.find((k) => k.type === Layout.Header);
  const rest = kids.filter((k) => k !== header);
  return <div><header>{header}</header><main>{rest}</main></div>;
}
Layout.Header = ({ children }: { children: React.ReactNode }) => <h1>{children}</h1>;

function AppHeader() { return <Layout.Header>Hi</Layout.Header>; }

<Layout><AppHeader /><p>Body</p></Layout>
```

Where does "Hi" end up in the DOM, and why does it not error?

P6.

```
class Socket {
  constructor() { console.log("open"); /* opens a connection */ }
  close() { console.log("close"); }
}
function Feed() {
  const [socket] = useState(() => new Socket());
  useEffect(() => () => socket.close(), [socket]);
  return null;
}
```

What is logged on mount in development StrictMode? In production? What does this imply about where the `open` side effect should live?

---

4. Part 4 — Debugging challenges (Debugging, 24 points)

For each: (a) state the symptom's root cause, (b) say how you would confirm it with DevTools or a test, (c) give the fix as code, (d) state how you would validate the fix. Six points: 2 root cause, 1 confirmation, 2 fix, 1 validation.

D1. `asChild` swallows handlers.

```
function Slot({ children, ...props }: React.HTMLAttributes<HTMLElement> & { children:
React.ReactElement }) {
  return React.cloneElement(children, props);
}
function Button({ asChild, ...props }: { asChild?: boolean } & React.ComponentProps<"button">)
{
  const Comp = asChild ? Slot : "button";
  return <Comp className="btn" onClick={() => track("button")} {...props} />;
}

// usage
<Button asChild ref={buttonRef}>
  <a href="/pay" onClick={() => track("link")} ref={linkRef}>Pay</a>
</Button>
```

Reported: "`track('link')` never fires, `linkRef.current` is `null` (while `buttonRef.current` points at the anchor), and clicking the link tracks `button` only."

D2. The input that loses focus.

```
function Page({ user }: { user: User }) {
  const SearchBox = withAnalytics(Search);          // HOC applied in render
  const value = { user, theme: "dark" };
  return (
    <ThemeContext value={value}>
      <SearchBox />
    </ThemeContext>
  );
}
```

Reported: "Every time `Page`'s parent re-renders, the search box loses text and focus, and all theme consumers re-render."

D3. The header shows another user's name.

```
// session.ts
export const session = { user: null as User | null };
export function setSession(u: User | null) { session.user = u; }

// Header.tsx
export function Header() { return <span>{session.user?.name ?? "Guest"}</span>; }
```

Server-rendered with Node. Reported: "In production, occasionally a request shows a different user's name. On the client, after `setSession(newUser)`, the header does not update until I navigate."

D4. The boundary that doesn't catch.

```
function Checkout() {
  async function pay() {
    const res = await fetch("/api/pay", { method: "POST" });
    if (!res.ok) throw new Error("payment failed");
  }
  return (
    <ErrorBoundary fallback={({ reset }) => <button onClick={reset}>Try again</button>}>
      <PayButton onClick={pay} />
      <Summary />   {/* throws during render if cart is corrupted */}
    </ErrorBoundary>
  );
}
```

Reported: "Payment failures show nothing and leave an unhandled rejection in the console. When `Summary` throws, the entire checkout is replaced by 'Try again', and clicking it immediately shows 'Try again' again."

---

5. Part 5 — Implementation exercises (Implementation, 24 points)

Six points each; see Part 7 for the rubric breakdown. Write real code, plus the tests described. TypeScript `strict`.

I1. Controllable state and a `Switch`. Implement `useControllableState<T>` and a `Switch` component (`role="switch"`) supporting `checked`, `defaultChecked`, and `onCheckedChange`. Requirements: controlled and uncontrolled modes; `onCheckedChange` called only on actual change; dev-only warning when the component changes mode; keyboard operable; ARIA state. Provide three tests: uncontrolled toggle, controlled with a parent that rejects changes, and mode-switch warning.

I2. A polymorphic `Text`. Implement `Text` with an `as` prop typed so that `<Text as="a" href="/x">` compiles, `<Text as="span" href="/x">` does not, `ref` works (React 19 `ref` as a prop), and the default element is `span`. List two limitations of your typing approach.

I3. An error boundary with reset keys and async routing. Implement a class `ErrorBoundary` with: `fallback({ error, reset })`, `onError(error, info)`, `resetKeys`. Implement a hook `useThrowToBoundary()` that lets event handlers and promises send an error to the nearest boundary. Specify what it can and cannot catch. Provide one test that proves `resetKeys` clears the error only when a key changes *after* an error was captured.

I4. DI + facade + command. Implement: (1) a `ServicesProvider` accepting `Partial<Services>` overrides over defaults, with a strict `useServices()` hook; (2) a facade hook `useInvoices()` returning `{ invoices, status, markPaid }`; (3) `markPaid` as a *command object* `{ type: "markPaid", id }` handled by a pure reducer for optimistic state with rollback on failure. Provide one test using fakes injected through the provider.

---

6. Part 6 — Architecture questions (Reasoning, 12 points)

Four points each: 1 for identifying forces/constraints, 1 for a defensible decision, 1 for trade-offs and a case where the decision would be wrong, 1 for risks/verification. Answers are paragraphs plus a diagram or list; no code required.

A1. Design-system `Select`. Your company's design system must ship a `Select` used by 12 teams on React 18 and 19, with form-library integration, async options, virtualized lists, custom option rendering, and strict accessibility. Choose the API shape (compound vs configuration props vs headless + styled layer), how state is owned, how consumers customize rendering (slots, render props, polymorphism), and your versioning/deprecation policy.

A2. Migrating a legacy app. A 300-component React 17-era app uses class containers, 40 HOCs, `defaultProps`, Redux for everything, and effect-based fetching. The business wants two features shipped monthly during the migration. Propose a plan: sequencing, which patterns you keep, which you retire, enforcement, metrics, and when you would stop.

A3. Data-layer seams. A product migrates its backend from REST to GraphQL over six months while keeping the UI shipping. Design the seams using adapter, strategy, factory, and dependency injection ideas (and which of them you decide *not* to use). Include error handling (typed errors, boundaries, retries), testing strategy, and what lives in the UI vs a services layer.

---

7. Part 7 — Detailed answer key and rubrics

> Attempt every question before reading this section.

7.1 Part 2 key (Explanation)

E1. *Sample.* Single Responsibility violation: a `ProductPage` fetches data, filters, formats prices, and renders markup; cost: any change (pricing format, API shape) edits the same file, tests need network mocks for formatting logic, and reuse is impossible. Dependency Inversion violation: a component imports `fetch("https://…")` directly; cost: tests, Storybook, and alternate backends require patching globals; fix by injecting a service through props or context. Open/Closed: adding a variant requires editing a growing `if` chain; fix with a union and an exhaustive switch or slot/composition. Over-engineering: Interface Segregation applied to a three-prop internal component by splitting props into five interfaces. Rubric: 2 = two principles with concrete React violations and costs plus a valid over-application; 1 = principles named with weak or no React cost; 0 = definitions only.

E2. The wrong abstraction arises when code is merged because it *looks* similar, then diverges; each divergence adds a parameter or flag, and the shared component becomes harder to change than the duplicates were. Duplication is cheaper while the cases are still discovering their differences (rule of three) because changing one copy cannot break another. Signals to abstract: the copies change together for the same reason, a bug fix has been applied to several copies, and the commonality is a *concept* (a domain rule), not coincidental structure. Rubric: 2 requires the "diverging parameters" mechanism and at least one signal.

E3. The docs say they found no use case where inheritance (extending component classes) is preferable to composition. Mechanisms: (1) `children`/slots/props for content customization (Card with header/footer); (2) specialization by rendering a generic component with specific props (`SignUpDialog` renders `Dialog`); (3) custom hooks for sharing logic; (4) context for ambient configuration. Each fits: slots for layout variation, specialization for preset behavior, hooks for logic reuse. Rubric: 2 = accurate claim, two mechanisms with fit.

E4. The separation isolates *data/side-effect acquisition* from *presentation*, giving testable and reusable views (Storybook, visual regression) and a seam for swapping data sources. It is not worth it for single-use components with trivial logic, where colocated hooks are simpler. Hooks changed the *mechanism* (a hook instead of a container component) not the concern. Server components relocate the container role to the server: data fetching in a server module, interactive presentation in a client component, with serialization limits on props crossing the boundary. Rubric: 2 = names the concern retained, a not-worth-it case, and the server relocation.

E5. A parent that inspects `children` only sees elements the consumer wrote at the top level; the sub-component renders later. Context lets any descendant, however wrapped, find the parent at render time. Breakages of inspection: wrapper components (type differs), fragments (`toArray` does not flatten them), `memo`/`lazy` changing the type, conditional rendering shapes, hot-reload identity changes. Rubric: 2 = context rationale plus two breakages.

E6. No. Hooks share *logic*; each call creates independent state in the calling component (`useCounter()` twice gives two counters). To share state, lift it, use context, or an external store. API design consequences: return the minimum (values and stable actions), document that each call is independent, avoid returning new object identities needlessly when consumers use them as dependencies. Rubric: 2 = correct claim, independence explanation, one API consequence.

E7. A headless component/hook supplies behavior (state, keyboard handling, ARIA) and leaves rendering to the consumer: prop getters, render props, or compound parts. It gives up out-of-the-box visuals and simple one-line usage, shifting styling and correct wiring to consumers (risk: consumers drop required ARIA attributes). It is not worth it when a single product needs one look (a styled component is simpler), or when team capacity to maintain markup correctness is low. Accessibility behavior (focus management, roving tabindex, `aria-*` relationships) is the hard part that headless libraries encapsulate. Rubric: 2 = definition, give-up, a not-worth-it scenario, a11y role.

E8. An inline object creates a new identity every provider render; context consumers re-render when the value changes by identity, so every provider render triggers every consumer even if the content is equal. Memoizing the value (`useMemo`) stabilizes identity when inputs are unchanged; splitting contexts by update frequency/domain (state vs actions, theme vs cart) narrows which consumers update. Throwing when the provider is missing converts silent default-value bugs into immediate, locatable errors. Note: context providers re-rendering does not re-render children passed as `children` (same element identity); only consumers. Rubric: 2 = identity mechanism and one mitigation and the strict-hook reason.

E9. Drawbacks: wrapper hell/DevTools noise; prop-name collisions and opaque injected props; typing difficulty (generic `Omit` gymnastics, casts); static properties and refs not forwarded unless hoisted/forwarded; order-dependent composition; creating HOCs inside render remounts. Still reasonable: wrapping components you cannot edit (third-party/legacy class components), cross-cutting instrumentation at a component boundary, or when a large codebase already uses the pattern and migration cost exceeds benefit. Rubric: 2 = three drawbacks with causes and one valid use.

E10. Both share behavior; hooks do so without extra tree nodes, with better composition (multiple hooks in one component) and typing. Render props are still needed when the *component owns where markup is placed* and passes per-item state into consumer-provided markup (virtualized rows, headless collection items), or inside class components. Rubric: 2 = comparison with the placement-of-markup criterion.

E11. Controlled: the parent owns the value via props and receives change notifications; uncontrolled: the component owns its state (initial `defaultValue`). Switching between modes triggers React's "changing an uncontrolled input to be controlled (or vice versa)" warning for form inputs because the component's source of truth changes. Reusable components detect mode by `value !== undefined`, keep internal state when uncontrolled, call `onChange` in both, and warn in development on mode changes. Rubric: 2 = definitions, the warning mechanism, and the dual-mode strategy.

E12. React may call reducers more than once (StrictMode double-invokes reducers and render functions in development; render may be discarded/restarted), and the reducer result must be a function of `(state, action)` so React can replay/rebase updates deterministically. Impurity (mutation, randomness, I/O) causes duplicated effects, lost updates, and inconsistent state. StrictMode's double invocation exposes it (e.g., items added twice). Rubric: 2 = replay/determinism reason plus StrictMode.

E13. A union of states makes invalid combinations (e.g., `loading` and `error` simultaneously, `data` present while `idle`) unrepresentable, and a transition function plus exhaustive checking forces handling of every state/event pair. It does not prevent logic errors within allowed transitions, race conditions in the effects that dispatch events (stale responses), or incorrect modeling (missing states). Rubric: 2 = unrepresentable states, exhaustiveness, and a limit.

E14. In concurrent rendering a render can be interrupted and resumed, and external mutable data can change between reads in different components of the same render ("tearing"). `useSyncExternalStore` makes React read a consistent snapshot and re-check it; it forces synchronous updates for store changes so UI is consistent. Effect-based subscriptions subscribe after commit and can miss changes or render inconsistent values. `getSnapshot` must return the same value (by `Object.is`) when the store has not changed, and must be pure and cheap; `getServerSnapshot` must match what the client first reads for hydration. Rubric: 2 = tearing explanation plus snapshot-stability requirement.

7.2 Part 3 key (Prediction)

P1. Dev StrictMode: `"Expensive render"` logs **twice at mount** (render is double-invoked in development; React DevTools can dim the second log), and **never again** during ticks. Production: once at mount. Reason: `Clock` re-renders on each state change, but `children` is the same element object created once in `App`, which did not re-render; React bails out of a child whose element identity (and thus props) are unchanged and has no pending work. This is the "children as props" optimization. Scoring: 1 for "not re-rendered on ticks", 1 for element-identity explanation (mount double render noted gets full credit but is not required).

P2. Development StrictMode: **two** `<li>` after one click. React invokes the reducer twice for the same state object in development; the first call pushes into `state.items` and returns a new wrapper; the second call (on the same original `state`) pushes again into the *same array*, so `items` has `["a","a"]`. Production: **one**. The code mutates state and "works" in production while hiding a bug. Fix: `return { items: [...state.items, action.item] }`. Scoring: 1 for 2 vs 1, 1 for the mutation + double-invocation explanation. (Executed on React 19.3.1: 2 `<li>` with `<StrictMode>`, 1 without.) The documented fact is that reducers must be pure.

P3. `name` is `undefined` initially, so the input is *uncontrolled*. On the first keystroke `setName("a")` makes `value` a string: React warns "A component is changing an uncontrolled input to be controlled" (wording may vary by version). The input still works in practice, but the mode switch is a bug. Fix: `useState("")`. Scoring: 1 for the warning, 1 for the cause (`undefined` value ⇒ uncontrolled).

P4. `getSnapshot` returns a new object on each call, never `Object.is`-equal to the previous. React sees "the store changed" every check; in development it warns that `getSnapshot` should be cached, and can result in an infinite re-render ("Maximum update depth exceeded"). The unused `store` constant is irrelevant. Fix: return `window.innerWidth` (primitive) or a cached immutable snapshot object that changes only when the width changes; also provide `getServerSnapshot` for SSR (`window` does not exist on the server). Scoring: 1 for identifying the unstable snapshot, 1 for the consequence/fix.

P5. The `<h1>Hi</h1>` appears inside `<main>`, with the `<p>`: `AppHeader`'s element type is `AppHeader`, not `Layout.Header`, so `find` returns `undefined`; `header` is `undefined`, `rest` contains both children. No error: `AppHeader` renders `Layout.Header`, which is a normal component and renders an `h1`

wherever it is placed. `<header>{undefined}</header>` renders an empty header. Scoring: 1 for location, 1 for the type-scanning reason.

P6. Development StrictMode logs `open`, `open`, `close`. The `useState` initializer runs twice (two `Socket`s are constructed; React keeps one result and discards the other, which is never closed: a leaked connection). Then StrictMode's simulated unmount/remount runs the effect cleanup for the kept socket (`close`) and re-runs the effect, which only registers the cleanup again; nothing reopens the socket, so the mounted component holds a **closed** socket. Production logs `open` once on mount and `close` on unmount. Conclusion: construction during render must be side-effect-free; create and open the connection *inside the effect* and close it in the cleanup, so the dev mount/unmount/mount cycle leaves a live connection and no leak. Scoring: 1 for the dev-vs-prod difference (open twice in dev, once in prod), 1 for the "side effects belong in effects with cleanup" conclusion. (Executed on React 19.3.1: with `<StrictMode>` the mount logged `open, open, close` and unmount added a second `close`; without it `open` then `close`.) The portable conclusion is what is graded.

7.3 Part 4 key (Debugging)

D1. *Root cause.* `cloneElement(children, props)` merges the given props over the child's: any prop passed from `Button` with the same name replaces the child's. `onClick` from `Button` replaces the link's `onClick`, so `track("link")` never fires. In React 19, `ref` is an ordinary prop, so `buttonRef` travels in `props` and `cloneElement` applies it as the element's ref, replacing the link's own `ref`; therefore `linkRef` is never attached (`null`) and `buttonRef` receives the anchor. A correct `Slot` must compose handlers and refs (in React 18 `ref` is not a prop and `Button` would need `forwardRef`; the same override logic applies). *Confirm.* Log in both handlers; inspect merged props in a debugger or React DevTools props panel for the `<a>`. *Fix.*

```
function mergeProps(slot: Record<string, any>, child: Record<string, any>) {
  const merged = { ...slot, ...child };
  for (const k of Object.keys(slot)) {
    if (/^on[A-Z]/.test(k) && typeof slot[k] === "function" && typeof child[k] === "function")
{
      merged[k] = (...a: unknown[]) => { child[k](...a); slot[k](...a); };
    } else if (k === "className") merged.className = [slot.className,
child.className].filter(Boolean).join(" ");
  }
  return merged;
}
function Slot({ children, ...slotProps }: any) {
  const child = React.Children.only(children) as React.ReactElement<any>;
  return React.cloneElement(child, { ...mergeProps(slotProps, child.props), ref:
composeRefs(slotProps.ref, child.props.ref) });
}
```

Validate. Test: click fires both handlers in order; `linkRef.current` is the anchor; `className` is merged; ref callbacks receive cleanup/null on unmount. Rubric: 2 root cause (handler override is mandatory; ref

reasoning with version awareness is credited), 1 confirmation, 2 fix (handlers + ref composition), 1 validation.

D2. *Root cause.* (1) `withAnalytics(Search)` creates a **new component type on every** `Page` **render**; React sees a different type at that position, unmounts the old subtree and mounts a new one, destroying input state and focus. (2) `value={{ user, theme: "dark" }}` creates a new object every render, so all consumers re-render whenever `Page` renders. *Confirm.* React DevTools Profiler "Why did this render" and component mount counts (the component re-mounts); add a `useEffect(() => console.log("mount"), [])` in `Search`; typing then re-render parent. *Fix.*

```
const SearchBox = withAnalytics(Search);          // module scope, created once

function Page({ user }: { user: User }) {
  const value = useMemo(() => ({ user, theme: "dark" as const }), [user]);
  return <ThemeContext value={value}><SearchBox /></ThemeContext>;
}
```

Even better, replace the HOC by a hook (`useAnalytics("search")`) inside `Search` . *Validate.* Test: type text, rerender parent, text persists and mount effect ran once; Profiler shows consumers skip when `user` is unchanged. Rubric: 2 root cause must include the remount mechanism (type identity); 1 confirmation; 2 fix (hoist + memo or hook); 1 validation.

D3. *Root cause.* Two problems. (1) Server: `session` is a **module-level mutable singleton** shared by all requests in the Node process; request A's `setSession` is visible to request B's render. This is a data leak. (2) Client: `Header` reads a mutable object during render without subscription; React is never told that `session.user` changed, so no re-render happens until something else renders (navigation). *Confirm.* Two concurrent requests with different users in a test; on the client, call `setSession` and observe no render in the Profiler. *Fix.* Per-request/app-instance store provided via context and read through `useSyncExternalStore` :

```
function createSessionStore(initial: User | null) {
  let user = initial;
  const ls = new Set<() => void>();
  return {
    get: () => user,
    set(u: User | null) { user = u; ls.forEach((l) => l()); },
    subscribe(l: () => void) { ls.add(l); return () => { ls.delete(l); }; },
  };
}
const SessionContext = createContext<ReturnType<typeof createSessionStore> | null>(null);

function SessionProvider({ initialUser, children }: { initialUser: User | null; children:
React.ReactNode }) {
  const [store] = useState(() => createSessionStore(initialUser));   // one per provider
instance
  return <SessionContext value={store}>{children}</SessionContext>;
}
function useSessionUser() {
  const store = use(SessionContext);
  if (!store) throw new Error("SessionProvider missing");
  return useSyncExternalStore(store.subscribe, store.get, store.get);
}
```

Pass `initialUser` from the request context (cookie/session lookup), never from module state. *Validate.* Concurrent-request test; client test that `store.set` updates the header; hydration test with matching snapshots. Rubric: 2 root cause (both problems named; one only = 1), 1 confirmation, 2 fix (per-request scope + subscription), 1 validation.

D4. *Root cause.* (1) Error boundaries catch errors thrown **during rendering, lifecycle methods, and constructors of descendants**; they do not catch errors in event handlers or async code (the `pay` rejection becomes an unhandled promise rejection). (2) `Summary` throws during render, so the boundary replaces the **whole** subtree, including the payment button: the boundary is too coarse. (3) "Try again" resets the boundary, but `Summary` re-renders with the same corrupted cart, throws again, and the boundary immediately shows the fallback: the reset has no changed input. *Confirm.* Reproduce: throw inside `pay` (no fallback); DevTools Console shows `Uncaught (in promise)`; use the Components tab to see the boundary's state; in the test, click reset twice and verify the same error. *Fix.*

```
function PayButton() {
  const [state, setState] = useState<{ status: "idle" | "paying" | "error"; message?: string
}>({ status: "idle" });
  async function pay() {
    setState({ status: "paying" });
    try {
      const res = await fetch("/api/pay", { method: "POST" });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setState({ status: "idle" });
    } catch (e) {
      setState({ status: "error", message: e instanceof Error ? e.message : "Payment failed"
});
    }
  }
  return <>
    <button onClick={pay} disabled={state.status === "paying"}>Pay</button>
    {state.status === "error" && <p role="alert">{state.message}</p>}
  </>;
}

<>
  <PayButton />
  <ErrorBoundary
    resetKeys={[cartVersion]}
    fallback={({ reset }) => <CartProblem onReload={() => { clearCorruptedCart(); reset(); }}
/>}>
    <Summary />
  </ErrorBoundary>
</>
```

Expected failures (payment) are data (state), not exceptions; unexpected render errors go to a *narrow* boundary with a reset path that changes the underlying cause (`resetKeys` , or a repair action). *Validate.* Test: failing `fetch` renders an alert and no unhandled rejection; corrupted cart renders only the cart fallback; after repair the summary renders. Rubric: 2 root causes (three listed; two minimum for full credit), 1 confirmation, 2 fix (state for expected errors + narrower boundary + meaningful reset), 1 validation.

7.4 Part 5 key (Implementation)

I1 solution.

```tsx
import { useCallback, useEffect, useRef, useState } from "react";

export function useControllableState<T>({
  prop, defaultProp, onChange,
}: { prop: T | undefined; defaultProp: T; onChange?: (v: T) => void }) {
  const [inner, setInner] = useState<T>(defaultProp);
  const isControlled = prop !== undefined;
  const value = (isControlled ? prop : inner) as T;

  const wasControlled = useRef(isControlled);
  useEffect(() => {
    if (process.env.NODE_ENV !== "production" && wasControlled.current !== isControlled) {
      console.warn(
        `A component changed from ${wasControlled.current ? "controlled" : "uncontrolled"} to
` +
        `${isControlled ? "controlled" : "uncontrolled"}. Decide for the component's
lifetime.`,
      );
    }
    wasControlled.current = isControlled;
  }, [isControlled]);

  const setValue = useCallback((next: T) => {
    if (!isControlled) setInner(next);
    if (!Object.is(next, value)) onChange?.(next);
  }, [isControlled, value, onChange]);

  return [value, setValue] as const;
}

type SwitchProps = Omit<React.ComponentPropsWithoutRef<"button">, "onChange" | "role" | "aria-
checked"> & {
  checked?: boolean;
  defaultChecked?: boolean;
  onCheckedChange?: (checked: boolean) => void;
};

export function Switch({ checked, defaultChecked = false, onCheckedChange, onClick, ...rest }:
SwitchProps) {
  const [on, setOn] = useControllableState({ prop: checked, defaultProp: defaultChecked,
onChange: onCheckedChange });
  return (
    <button
      type="button"
      {...rest}
      role="switch"
```

```
      aria-checked={on}
      data-state={on ? "on" : "off"}
      onClick={(e) => { onClick?.(e); if (!e.defaultPrevented) setOn(!on); }}
    />
  );
}
```

Notes: `<button>` handles Space/Enter natively (keyboard operable). `role` and `aria-checked` come after `{...rest}` so consumers cannot break semantics (documented decision). In a controlled parent that rejects changes, `setOn` calls `onCheckedChange(next)` but `value` stays `prop`; the visual state follows the prop. Tests: (1) uncontrolled: click toggles `aria-checked`; (2) controlled `checked={false}` with a no-op handler: click calls the handler with `true`, `aria-checked` remains `false`; (3) rerender from `checked={undefined}` to `checked={true}` triggers `console.warn` (spy). Rubric (6): mode detection by `!== undefined` (1), uncontrolled internal state (1), `onChange` only on actual change (1), dev warning without side effects in render (1), accessible semantics and keyboard (1), tests covering the three cases (1). Common loss: warning implemented by calling `console.warn` during render; or `onChange` fired in an effect.

I2 solution.

```
import type { ComponentPropsWithRef, ElementType, ReactNode } from "react";

type TextOwnProps<E extends ElementType> = { as?: E; children?: ReactNode };
type TextProps<E extends ElementType> = TextOwnProps<E> &
  Omit<ComponentPropsWithRef<E>, keyof TextOwnProps<E>>;

export function Text<E extends ElementType = "span">({ as, ...props }: TextProps<E>) {
  const Comp: ElementType = as ?? "span";
  return <Comp {...props} />;
}

// <Text as="a" href="/x" ref={anchorRef}>ok</Text>      compiles
// <Text as="span" href="/x">no</Text>                    error: 'href' not in span props
// <Text>default span</Text>
```

Limitations: (1) `Omit` over large prop unions/intersections can be slow to type-check and loses distributive behavior for union components; (2) when `as` is a custom component with required props, inference may need explicit generics; the implementation requires an internal cast (`Comp: ElementType`) so the type safety exists only at the call sites; also `ref` typing differs between React 18 (`forwardRef` needed) and 19. Rubric (6): generic with default (1.5), correct prop derivation using `ComponentPropsWithRef` and omission of own props (1.5), `ref` works (1), the three type behaviors demonstrated (1), two valid limitations (1).

I3 solution.

```tsx
import { Component, useState, type ErrorInfo, type ReactNode } from "react";

type FallbackArgs = { error: Error; reset: () => void };
type Props = {
  children: ReactNode;
  fallback: (a: FallbackArgs) => ReactNode;
  onError?: (error: Error, info: ErrorInfo) => void;
  resetKeys?: readonly unknown[];
};
type State = { error: Error | null };

const changed = (a: readonly unknown[] = [], b: readonly unknown[] = []) =>
  a.length !== b.length || a.some((x, i) => !Object.is(x, b[i]));

export class ErrorBoundary extends Component<Props, State> {
  state: State = { error: null };

  static getDerivedStateFromError(error: Error): State { return { error }; }

  componentDidCatch(error: Error, info: ErrorInfo) { this.props.onError?.(error, info); }

  componentDidUpdate(prevProps: Props, prevState: State) {
    // Only reset if an error was ALREADY showing before this update; otherwise an update that
    // both changes keys and throws would clear the freshly captured error immediately.
    if (this.state.error !== null && prevState.error !== null && changed(prevProps.resetKeys,
this.props.resetKeys)) {
      this.reset();
    }
  }

  reset = () => this.setState({ error: null });

  render() {
    return this.state.error ? this.props.fallback({ error: this.state.error, reset: this.reset
}) : this.props.children;
  }
}

export function useThrowToBoundary() {
  const [, setState] = useState();
  return (error: unknown) => {
    setState(() => { throw error instanceof Error ? error : new Error(String(error)); });
  };
}
```

What `useThrowToBoundary` can catch: any error you explicitly route (event handler `try/catch`, promise `.catch`). It cannot catch errors you never route, errors thrown after unmount (no state update occurs), or errors outside the React tree; it also converts an *expected* failure into a UI-wide fallback, so use it only for unexpected failures. Test: render a child that throws when `shouldThrow`; assert fallback; change `resetKeys` and `shouldThrow` simultaneously, assert the error is *not* cleared by the same update that captured it (this is the `prevState.error !== null` guard); change a key later with `shouldThrow=false`, assert recovery. Rubric (6): class with `getDerivedStateFromError` (1), `componentDidCatch` reporting (1), reset by keys with guard (1.5), hook that rethrows during a state update (1), correct limits statement (1), test (0.5). ⚠ Error boundaries remain class-only in React 19; verify current docs. Production code may use the `react-error-boundary` library instead.

I4 solution (sketch).

```
type Invoice = { id: string; total: number; paid: boolean };
type Services = {
  invoices: { list(): Promise<Invoice[]>; markPaid(id: string): Promise<void> };
};

const defaults: Services = {
  invoices: {
    list: async () => (await fetch("/api/invoices")).json(),
    markPaid: async (id) => { const r = await fetch(`/api/invoices/${id}/pay`, { method:
"POST" }); if (!r.ok) throw new Error(String(r.status)); },
  },
};

const ServicesContext = createContext<Services | null>(null);

export function ServicesProvider({ overrides, children }: { overrides?: Partial<Services>;
children: React.ReactNode }) {
  const value = useMemo(() => ({ ...defaults, ...overrides }), [overrides]);
  return <ServicesContext value={value}>{children}</ServicesContext>;
}
export function useServices(): Services {
  const s = use(ServicesContext);
  if (!s) throw new Error("useServices must be used inside <ServicesProvider>");
  return s;
}

type Command = { type: "markPaid"; id: string };
type State = { invoices: Invoice[]; status: "loading" | "ready" | "error" };
type Action =
  | { type: "loaded"; invoices: Invoice[] }
  | { type: "failed" }
  | { type: "apply"; command: Command }
  | { type: "rollback"; command: Command };

function reducer(s: State, a: Action): State {
  switch (a.type) {
    case "loaded": return { status: "ready", invoices: a.invoices };
    case "failed": return { ...s, status: "error" };
    case "apply": return { ...s, invoices: s.invoices.map((i) => i.id === a.command.id ? {
...i, paid: true } : i) };
    case "rollback": return { ...s, invoices: s.invoices.map((i) => i.id === a.command.id ? {
...i, paid: false } : i) };
  }
}

export function useInvoices() {
```

```
  const { invoices: api } = useServices();
  const [state, dispatch] = useReducer(reducer, { invoices: [], status: "loading" });

  useEffect(() => {
    let ignore = false;
    api.list().then(
      (invoices) => { if (!ignore) dispatch({ type: "loaded", invoices }); },
      () => { if (!ignore) dispatch({ type: "failed" }); },
    );
    return () => { ignore = true; };
  }, [api]);

  const markPaid = useCallback(async (id: string) => {
    const command: Command = { type: "markPaid", id };
    dispatch({ type: "apply", command });
    try { await api.markPaid(id); }
    catch { dispatch({ type: "rollback", command }); }
  }, [api]);

  return { invoices: state.invoices, status: state.status, markPaid };
}
```

Test: wrap in `<ServicesProvider overrides={{ invoices: fake }}>`; fake `list` resolves two invoices; fake `markPaid` rejects; click "pay"; assert the row flips to paid then back to unpaid, with no network. Rubric (6): strict hook + partial overrides (1.5), facade returns a narrow API (1), command as data with pure reducer (1.5), rollback correctness (1), test via injected fake (1). Known limits to acknowledge for full marks: the rollback sets `paid:false` rather than restoring the *previous* value (fine here, wrong for general fields; store the inverse command or snapshot), `overrides` must be a stable reference (or memoized by the caller) else `value` changes each render, and concurrent commands on the same id can interleave. The `markPaid` function should only be called from within an Action/handler, not render.

7.5 Part 6 key (Architecture)

A1. Evaluation criteria and example answer. Forces: 12 teams, mixed React versions (18/19: so `ref` -as-prop and `<Context>` shorthand cannot be assumed; `forwardRef` stays), a11y strictness, form integration (controlled/uncontrolled and `name` / `form` participation), async/virtualized lists (state ownership and performance), custom rendering, long-lived API promises. *Example decision:* two layers: (1) a **headless core** (`useSelect` returning state, prop getters, keyboard/ARIA behavior, controllable state) with no styling, internal and optionally public; (2) a **styled, compound public API** (`Select.Root/Trigger/Content/Option/Group`) built on it, exposing `value/defaultValue/onValueChange` (dual mode), `open/defaultOpen/onOpenChange` , option content via `children` (slots) and `renderValue` render prop for the trigger, `asChild` on parts for polymorphism, `data-*` state attributes and CSS variables for styling hooks. Async options via an `options` adapter prop or an `onSearch` callback; virtualization as an optional `virtualized` content part that exposes item measurement hooks instead of baking a library in. Versioning: public surface list (props, DOM roles, data attributes, tokens), API-surface tests, deprecation with dev warnings for one minor, codemods for majors, peer ranges `react: ^18 || ^19` , `exports` map, `sideEffects` correctness. Trade-offs: compound API

has a steeper learning curve and larger surface than config props; headless increases consumer burden. Wrong when: only one team and one look exist (a styled configuration-prop component is cheaper). Risks/verification: contract tests under both React versions, SSR/hydration test (`useId`), a11y tests (axe plus manual screen reader), bundle size budget. Rubric: forces (1), layered decision with ownership/customization specifics (1), trade-offs and counter-scenario (1), versioning/testing/risk (1).

A2. Example answer. Forces: two features/month (no freeze), 300 components, mixed patterns, risk of regressions, team learning curve. Plan: (0) **safety net**: characterization tests on top revenue/critical flows, error monitoring, performance baselines; (1) **freeze growth**: lint rules forbidding new HOCs, class components, `defaultProps`, and effect-fetching; codemods for mechanical changes (`defaultProps` → default params, `forwardRef` only when upgrading to 19); (2) **upgrade path**: move to React 18 then 19 as required (use the upgrade guides; verify codemods), enabling StrictMode per area; (3) **strangler by feature**: when a feature is touched for product work, migrate its container/HOC/data access to hooks plus a server-state library at the boundary; keep Redux for truly global/complex client state, retire it for server data; (4) **retire last**: HOCs that wrap third-party/unowned components and stable low-change legacy screens; (5) **metrics**: counts of banned patterns (trend to zero in migrated dirs), render counts in key flows, bundle size, defect rate, PR cycle time; (6) **stop criteria**: stop migrating areas with no change in a year; budget ~15–20 % of capacity; review quarterly. Trade-offs: partial migration leaves two idioms to support (cost); wrong when the app will be replaced within a year (invest in tests and monitoring only). Risks: hidden coupling via statics, context behavior changes, performance regressions after removing memoization. Rubric: constraint awareness (1), sequenced strategy with retained/retired patterns (1), enforcement and metrics (1), stop criteria/risk/counter-case (1).

A3. Example answer. Seam design: UI components depend on **domain-shaped hooks/services** (`useOrders()`, `OrdersService`), not on transport. An **adapter** per backend (`restOrdersAdapter`, `graphqlOrdersAdapter`) maps wire formats to the domain model (`Order`), including normalization of field names, enums, money, and error types. A **factory** (`createServices(config)`) chooses adapters per feature flag or environment; **strategy** is used where the *behavior* varies by runtime choice (e.g., pagination mode), not just transport; **DI via context** provides `Services` at the app root and in tests, scoped per request on the server. Not used: a heavy DI container or class hierarchies (no need), a universal "repository" for every entity (YAGNI). Error handling: adapters translate transport errors into **typed domain errors** (`NotFound`, `Unauthorized`, `Validation`, `Network`), expected failures are returned as data to forms/actions, unexpected ones propagate to boundaries at route/feature granularity with reset keys; retries live in the data layer (idempotent reads only) and are reported. Migration: dual-run per feature: both adapters implemented behind the same interface, contract tests run against both (shared test suite), flag rollout by percentage, compare results in shadow mode for risky reads, remove REST adapters last. UI vs services: the UI keeps presentation and interaction state; caching/invalidation sit at the server-state layer (library) which calls the services. Trade-offs: adapter layer is overhead if the migration is small; wrong if the GraphQL schema is the domain model (then adapters add little). Risks: lowest-common-denominator interface hiding GraphQL's selection benefits; leaking wire types; N+1 queries. Rubric: seam identification and patterns chosen/rejected with reasons (1), error and testing strategy (1), migration mechanics (1), trade-offs/counter-case/risks (1).

---

8. Part 8 — Scoring rubric and promotion criteria

8.1 Score sheet

Category	Items	Max	Your score	%	Threshold (70 %)
Explanation	E1–E14	28			19.6 → **20**
Reasoning	P1–P6 (12) + A1–A3 (12)	24			16.8 → **17**
Debugging	D1–D4	24			16.8 → **17**
Implementation	I1–I4	24			16.8 → **17**
Overall		**100**			**≥ 80**

8.2 Scoring guidance

- **Explanation (2/1/0)** per the rubric under each key entry. Give 1 for correct statements without the causal mechanism; withhold credit for statements that contradict the lesson (for example "hooks share state").
- **Prediction (P)**: 1 for the right observable result, 1 for the right cause. A right result with a wrong cause earns 1 and indicates a fragile mental model: treat it as a gap.
- **Architecture (A)**: grade on the four criteria in Part 6 and the sample's rubric. A different decision with sound forces and trade-offs earns full credit. An answer that picks a fashionable pattern without stating constraints earns at most 2.
- **Debugging (D)**: root cause (2), confirmation method (1), fix (2), validation (1). A fix that works but does not address the root cause (for example, wrapping everything in `memo` to hide remounts) earns at most 1 for the fix.
- **Implementation (I)**: use the per-task breakdown. Code that does not typecheck loses 1 point from the task; code that passes only the happy path loses 1–2 depending on the missing edge cases listed.
- **Integrity check**: if you looked at the key mid-question, mark that question 0 for scoring, then study it. The assessment measures independent recall and reasoning.

8.3 Promotion criteria (to Phase 6 and Project 7)

Promote when **all** are true:

1. Overall score ≥ 80 / 100.
2. Each category ≥ 70 % (≥ 20 / 17 / 17 / 17 as in the table).
3. You can build, without reference, a compound component with context-wired ARIA, a dual-mode (controlled/uncontrolled) hook, and an error boundary with reset keys (I1, I3 and the lesson 05 exercises), *and* explain each design choice.
4. You can take one legacy component from lesson 25's labs that you have not seen the solution to and refactor it with a stated force, a small-step plan, and verification.

If you meet 1–2 but not 3–4, you may start Phase 6 (data fetching) but must complete Project 7 — Reusable component library before Phase 8 (testing) and Phase 10 (design-system governance).

8.4 Remediation map

Weak area (category below 70 % or repeated misses)	Re-study	Then redo
SOLID/DRY/composition (E1–E3)	01, 02, 03	E1–E3; A2
Component structure (E4–E7, P5)	04, 05, 06, 07, 18	E4–E7; I4 facade part
Provider/HOC/render props (E8–E10, D2)	08, 09, 10	E8–E10, D2
State patterns (E11–E14, P2–P4, I1)	11, 12, 13, 14	P2–P4, I1
Boundaries and error handling (D4, I3)	21	D4, I3
Slots/polymorphism/libraries (D1, I2, A1)	19, 20, 22	D1, I2, A1
Adapter/DI/facade/command (I4, A3)	15, 16, 17	I4, A3
Modern API shifts and OOP mapping (P6, D3, A2)	23, 24	P6, D3, A2
Refactoring discipline (all)	25	One lab from scratch

8.5 Retake policy

Wait at least 3 days after remediation. Rewrite the weak categories using *new variants* rather than re-reading the same questions: take a component from your own work, state the force, refactor it, and write the questions an interviewer would ask about it. Retake only the categories below threshold; the overall score is recalculated with your best passing attempt per category.

---

9. After the assessment

You have finished Phase 5. Continue with Phase 6 — Data fetching and state ecosystems, where the data-layer seams from A3, the error handling patterns from 21, and the state patterns from 12–14 become concrete tools (TanStack Query, Redux Toolkit, Zustand). In parallel, build Project 7 — Reusable component library to turn Phase 5's API-design patterns into a shipped, versioned package.